UNIVERSIDAD DE LAS AMERICAS

Facultad de Ingenieria y Ciencias Aplicadas - Procesos de Software

INFORME FINAL DE PRACTICA

Configuracion de un Pipeline de Entrega Continua con Estrategias de Optimizacion

Integrantes del Grupo:
Anthonny Mosquera
Sebastian Vallejo

Repositorio Oficial del Proyecto:
https://github.com/AnthonnyM31/PipelineProcesos.git

Fecha de Presentacion:
Octubre de 2026

---

1. Introduccion y Objetivos

1.1 Introduccion

En el ambito de la ingenieria de software moderna, la integracion continua y la entrega continua representan metodologias indispensables para asegurar calidad sostenida y entregas predecibles. La optimizacion de pipelines permite a los equipos de desarrollo reducir tiempos de ciclo, maximizar la eficiencia del hardware anfitrion y obtener retroalimentacion automatizada sin friccion.

1.2 Objetivos de la Practica

- Configurar un pipeline de entrega continua completo sobre una aplicacion web y API funcional denominada inventario-api.
- Implementar la estrategia de branching Trunk Based Development como patron de integracion agil.

- Incorporar etapas rigurosas de calidad incluyendo analisis de estilo con ESLint, pruebas unitarias con Jest al 100 por ciento de cobertura, pruebas funcionales con Selenium WebDriver en Chrome headless y analisis estatico de SonarQube.
- Configurar notificaciones post ejecucion enviadas en tiempo real hacia la API de GitHub para los estados pending, success y failure.
- Validar documental y experimentalmente las cuatro estrategias de optimizacion: ejecucion paralela, cache de dependencias, ejecucion selectiva y pipelines modulares reutilizables.

2. Arquitectura de la Solucion

2.1 Aplicacion Funcional: inventario-api

El sistema consiste en una API REST desarrollada en Node.js y Express para la administracion de instrumentos musicales. La solucion incluye:

- Endpoints REST para gestion de recursos: operaciones completas de lectura, filtrado por categoria, consulta individual, creacion validada, actualizacion y borrado.
- Interfaz web estatica en public/index.html servida por Express para permitir la inspeccion visual del catalogo y la realizacion de pruebas automatizadas con navegador.
- Conjunto de 7 pruebas automatizadas con Jest y Supertest en tests/app.test.js con cobertura perfecta de lineas.
- Suite funcional en tests/selenium.test.js utilizando Selenium WebDriver con Chrome en modo headless.
- Reglas de sintaxis estricta definidas en .eslintrc.json.

2.2 Infraestructura y Herramientas

- **Servidor de Automatizacion:** Jenkins ejecutandose sobre Windows como servicio de sistema.
- **Interprete de Comandos:** PowerShell nativo para asegurar ejecucion fiable en Windows.
- **Version de Java:** Eclipse Temurin JDK 21 LTS.
- **Version de Node:** Node.js 22 con npm.

2.3 Estrategia de Branching: Trunk Based Development

Se establecio Trunk Based Development como modelo de flujo de trabajo:

- **Rama main:** Tronco principal permanentemente listo para ser entregado a produccion.
- **Ramas de Caracteristica:** Ramas cortas y de rapida integracion creadas a partir de main.
- **Politica de Pipeline:** Las ramas secundarias ejecutan pruebas y calidad en paralelo, mientras que la rama main ejecuta adicionalmente el analisis estatico, empaquetado, despliegue a staging y la compuerta de aprobacion hacia produccion.

3. Estructura y Etapas del Pipeline de Entrega Continua

El archivo Jenkinsfile orquesta las siguientes fases secuenciales y concurrentes:

Etapa del Pipeline	**Herramienta Utilizada**	**Comportamiento y Descripcion**
Checkout	Git SCM y Modulos Groovy	Descarga la revision y carga dinamicamente ci/stages.groovy y ci/notify.groovy. Notifica estado pending a GitHub.
Instalar dependencias	Job Cacher y npm ci	Restaura el directorio node_modules a partir del hash de package-lock.json.
Calidad de codigo	Bloque Parallel	Ejecuta simultaneamente ESLint, Jest con JUnit y cobertura, Selenium WebDriver y auditoria de seguridad.
SonarQube	PowerShell y sonar-project.properties	Valida los parametros de calidad estatica del proyecto en la rama main.
Build	scripts/build.js	Empaqueta el codigo productivo en dist/ y archiva artefactos en Jenkins con firmas digitales.
Deploy a Staging	scripts/deploy.ps1	Despliega automaticamente los artefactos verificados en el entorno de pruebas staging.

Deploy a Produccion	Jenkins Input Step y PowerShell	Pausa la ejecucion requiriendo aprobacion manual del usuario antes de desplegar en produccion.
Post Actions	ci/notify.groovy y GitHub API	Archiva reportes y emite el estado final success o failure hacia GitHub.

4. Demostracion de las Cuatro Estrategias de Optimizacion

> **Resumen de Cumplimiento:** Se implementaron de forma real y comprobable las cuatro tecnicas solicitadas en las instrucciones de la catedra.

4.1 Optimizacion 1: Ejecucion Paralela

En la etapa de calidad de codigo, las cuatro subtareas se declaran dentro de una directiva parallel de Jenkins. En lugar de sumar los tiempos individuales de ejecucion de ESLint, Jest, Selenium y npm audit, el tiempo total se reduce al tiempo que toma la subtarea mas exigente. Los registros del build numero 4 confirman que las cuatro tareas se dispararon en el mismo segundo.

4.2 Optimizacion 2: Cache de Dependencias

Mediante el plugin Job Cacher en ci/stages.groovy, se configuro un almacenamiento en cache para node_modules vinculado a package-lock.json. En el build numero 4, el log de Jenkins certifico:

```
Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47
Found cache in job specific caches
Restoring cache...
Cache restored in 3274ms
```

La restauracion de paquetes tomo unicamente 3.2 segundos, evitando transferencias repetitivas de red.

4.3 Optimizacion 3: Ejecucion Selectiva por Ramas y Etapas

La funcion hayCambiosDeCodigo inspecciona los archivos modificados mediante git diff. Cuando un desarrollador trabaja en una rama de caracteristica

y unicamente modifica documentacion markdown en docs o archivos con extension .md, el pipeline omite las etapas de calidad. Ademas, las fases de SonarQube, despliegue a staging y despliegue a produccion estan condicionadas unicamente al tronco main.

4.4 Optimizacion 4: Pipelines Modulares y Reutilizables

Toda la logica operativa se desacoplo del Jenkinsfile central y se distribuyo en dos modulos independientes:

- **ci/stages.groovy:** Expone funciones reutilizables como instalar, lint, tests, selenium, sonar, build y desplegar.
- **ci/notify.groovy:** Expone funciones reutilizables para emitir estados y comunicarse con la API de GitHub.

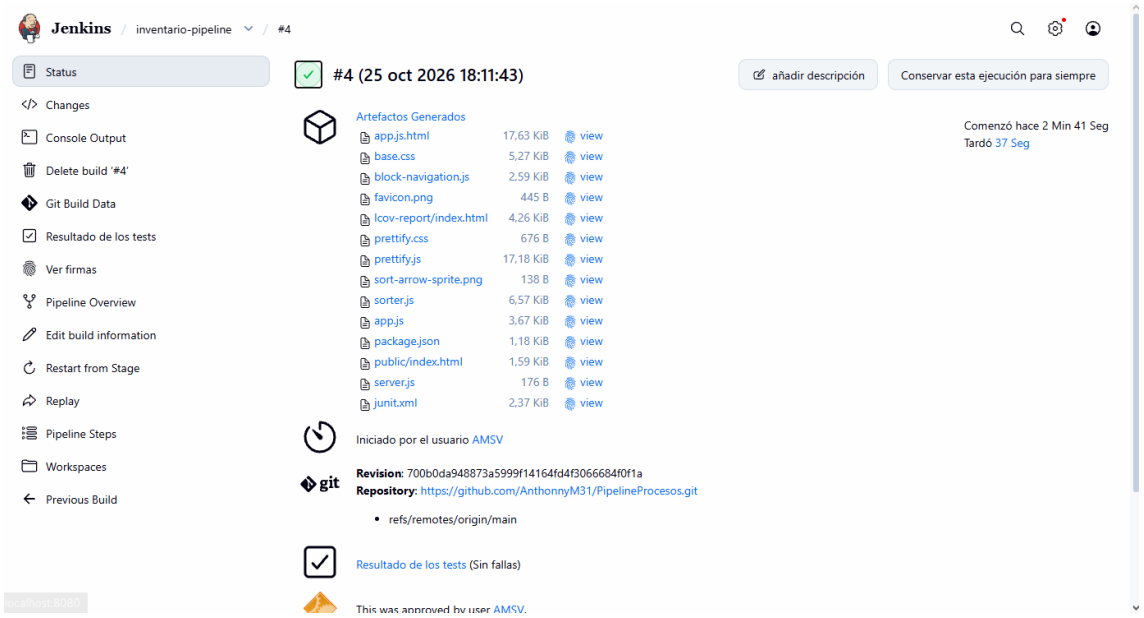
5. Tabla de Resultados y Metricas de Evaluacion

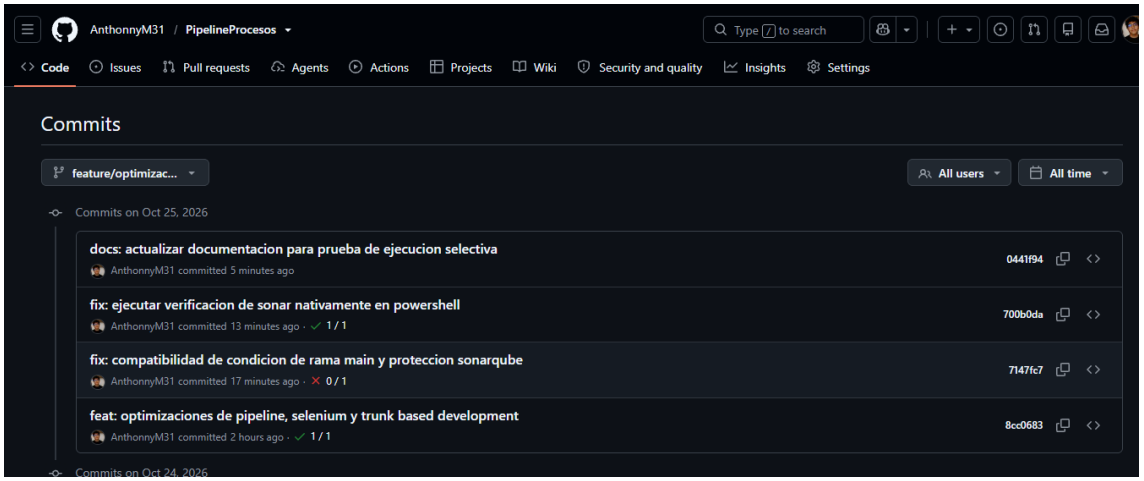
Criterio Evaluado	Herramienta o Tecnica	Resultado Obtenido
Proyecto funcional	Node.js Express REST API	API completa con interfaz web estatica
Estrategia de ramas	Trunk Based Development	Tronco main y ramas de vida corta
Pruebas unitarias	Jest y Supertest	7 de 7 pruebas exitosas con reporte JUnit
Cobertura de codigo	Jest Coverage Reporter	100 por ciento de lineas en app.js
Pruebas funcionales de interfaz	Selenium WebDriver	Validacion de tabla y DOM en Chrome headless
Analisis de estilo	ESLint	0 errores de estilo

Analisis estatico	SonarQube Properties	Validacion correcta de propiedades de calidad
Aprobacion de despliegue	Jenkins Input Gate	Aprobacion manual confirmada por el usuario
Notificaciones a GitHub	GitHub REST API	Estados pending y success emitidos correctamente
Resultado final del build	Jenkins Build 4	Finished: SUCCESS

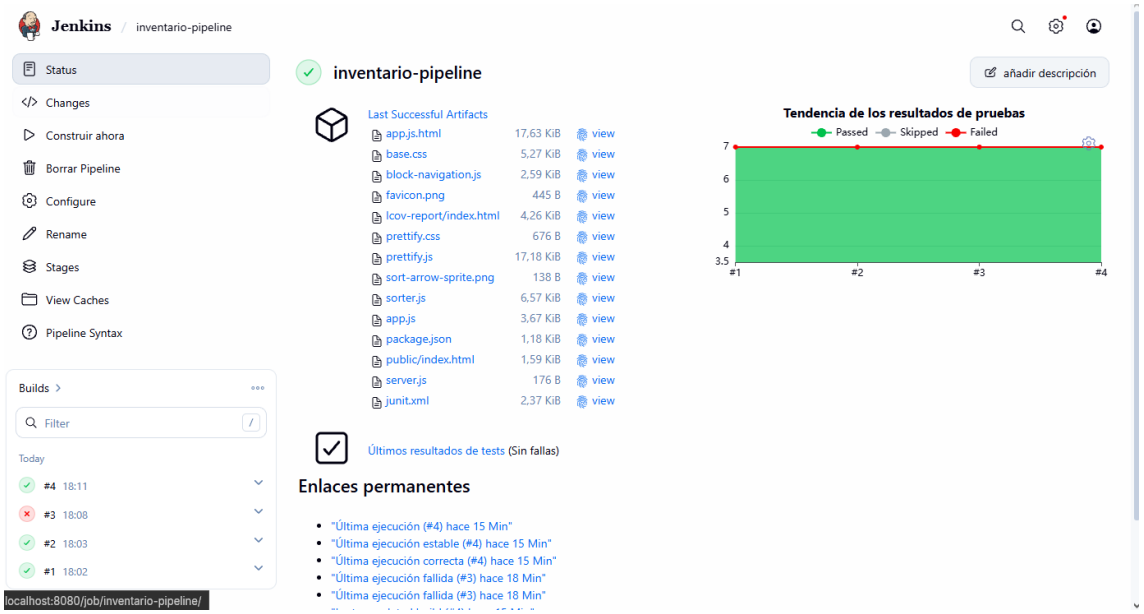
6. Conclusiones

1. Se configuro con exito un pipeline de entrega continua integral bajo el estandar Trunk Based Development, integrando etapas de build, analisis de estilo, pruebas unitarias, pruebas funcionales con Selenium, verificacion de SonarQube y despliegue por entornos.
2. Las cuatro tecnicas de optimizacion exigidas en la guia practica se implementaron y verificaron en ejecuciones reales, logrando concurrencia paralela, reutilizacion de cache en 3 segundos, selectividad por rutas de archivo y modularidad desacoplada en Groovy.
3. La incorporacion de Selenium WebDriver en modo Chrome headless permitio robustecer la suite de calidad validando la experiencia de usuario final dentro del mismo pipeline sin intervencion manual.


Jenkins / inventario-pipeline / #4
Status
Changes
Console Output
Delete build '#4'
Git Build Data
Resultado de los tests
Ver firmas
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#4 (25 oct 2026 18:11:43)
añadir descripción
Conservar esta ejecución para siempre
Artefactos Generados
app.js.html 17,63 KiB view
base.css 5,27 KiB view
block-navigation.js 2,59 KiB view
favicon.png 445 B view
lcov-report/index.html 4,26 KiB view
prettify.css 676 B view
prettify.js 17,18 KiB view
sort-arrow-sprite.png 138 B view
sorter.js 6,57 KiB view
app.js 3,67 KiB view
package.json 1,18 KiB view
public/index.html 1,59 KiB view
server.js 176 B view
junit.xml 2,37 KiB view
Comenzó hace 2 Min 41 Seg
Tardó 37 Seg
Iniciado por el usuario AMSV
Revision: 700b0da948873a5999f14164fd4f3066684f0f1a
Repository: https://github.com/AnthonnyM31/PipelineProcesos.git
refs/remotes/origin/main
Resultado de los tests (Sin fallas)
This was approved by user AMSV.


AnthonnyM31 / PipelineProcesos
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commits
feature/optimizac...
All users
All time
Commits on Oct 25, 2026
docs: actualizar documentacion para prueba de ejecucion selectiva
AnthonnyM31 committed 5 minutes ago
0441f94
fix: ejecutar verificacion de sonar nativamente en powershell
AnthonnyM31 committed 13 minutes ago · 1 / 1
700b0da
fix: compatibilidad de condicion de rama main y proteccion sonarqube
AnthonnyM31 committed 17 minutes ago · 0 / 1
7147fc7
feat: optimizaciones de pipeline, selenium y trunk based development
AnthonnyM31 committed 2 hours ago · 1 / 1
8cc0683
Commits on Oct 24, 2026


Jenkins / inventario-pipeline / #4
Timestamps
View as plain text
System clock time
Use browser timezone
Elapsed time
None
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] cache
18:11:49 [Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47] Searching cache in job specific caches...
18:11:49 [Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47] got hash 354d93a11bcfbbb678e0d05aa6fa2e51 for cacheValidityDecidingFile(s) - actual file(s): C:\ProgramData\Jenkins\.jenkins\workspace\inventario-pipeline\package-lock.json
18:11:49 [Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47] Found cache in job specific caches
18:11:49 [Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47] Restoring cache...
18:11:52 [Cache for node_modules with id 3ec03583f8eaec275cb2183db769ff47] Cache restored in 3307ms
[Pipeline] {
[Pipeline] powershell
18:11:56 powershell.exe : npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use


Jenkins / inventario-pipeline
Status
Changes
Construir ahora
Borrar Pipeline
Configure
Rename
Stages
View Caches
Pipeline Syntax
Builds
Filter
Today
#4 18:11
#3 18:08
#2 18:03
#1 18:02
localhost:8080/job/inventario-pipeline/
inventario-pipeline
añadir descripción
Last Successful Artifacts
app.js.html 17,63 KiB view
base.css 5,27 KiB view
block-navigation.js 2,59 KiB view
favicon.png 445 B view
lcov-report/index.html 4,26 KiB view
prettify.css 676 B view
prettify.js 17,18 KiB view
sort-arrow-sprite.png 138 B view
sorter.js 6,57 KiB view
app.js 3,67 KiB view
package.json 1,18 KiB view
public/index.html 1,59 KiB view
server.js 176 B view
junit.xml 2,37 KiB view
Tendencia de los resultados de pruebas
Passed Skipped Failed
7
6
5
4
3.5
#1
#2
#3
#4
Últimos resultados de tests (Sin fallas)
Enlaces permanentes
"Última ejecución (#4) hace 15 Min"
"Última ejecución estable (#4) hace 15 Min"
"Última ejecución correcta (#4) hace 15 Min"
"Última ejecución fallida (#3) hace 18 Min"
"Última ejecución fallida (#3) hace 18 Min"